Lab 9 - Decision Trees and Random Forests

ARTI308 - Machine Learning | LendingClub Loan Classification

Completed solution using the attached **02 - Decision Trees and Random Forest Project** and **loan_data.csv**. The original project questions are preserved in the solved Jupyter Notebook.

1. Import libraries and load data

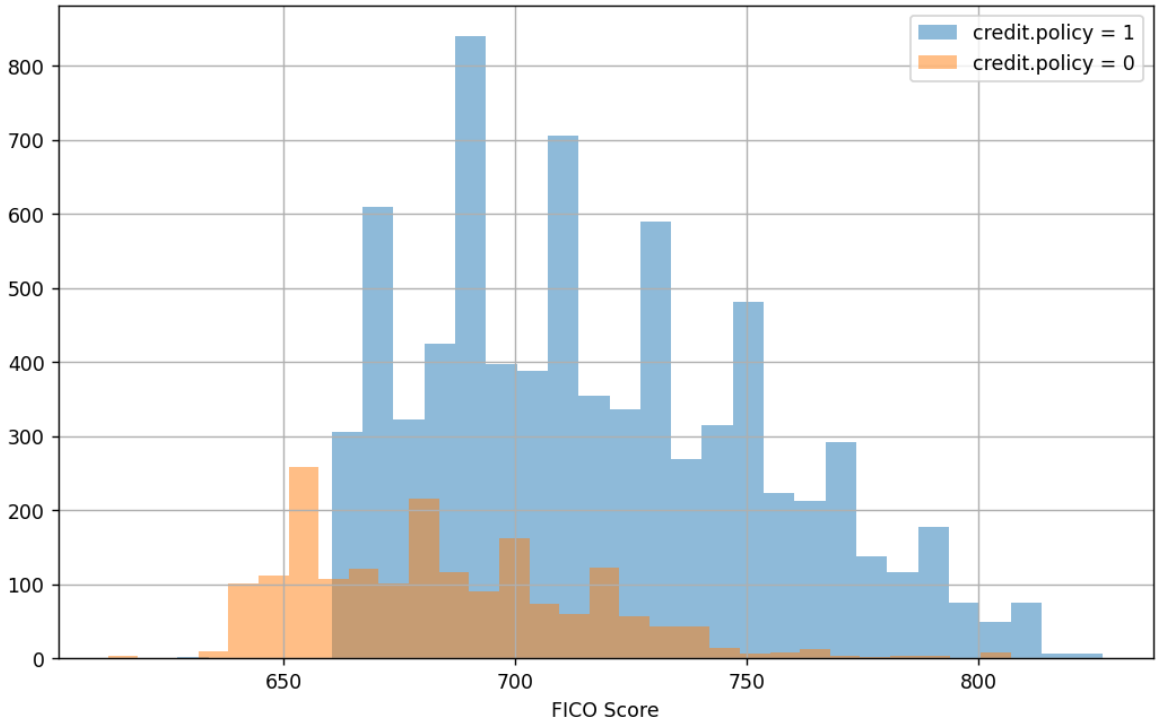
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
loans = pd.read_csv('loan_data.csv')
loans.info()
loans.head()
loans.describe()
```

The dataset contains **9,578 rows** and **14 columns**, with **0 missing values**. The target column is **not.fully.paid**.

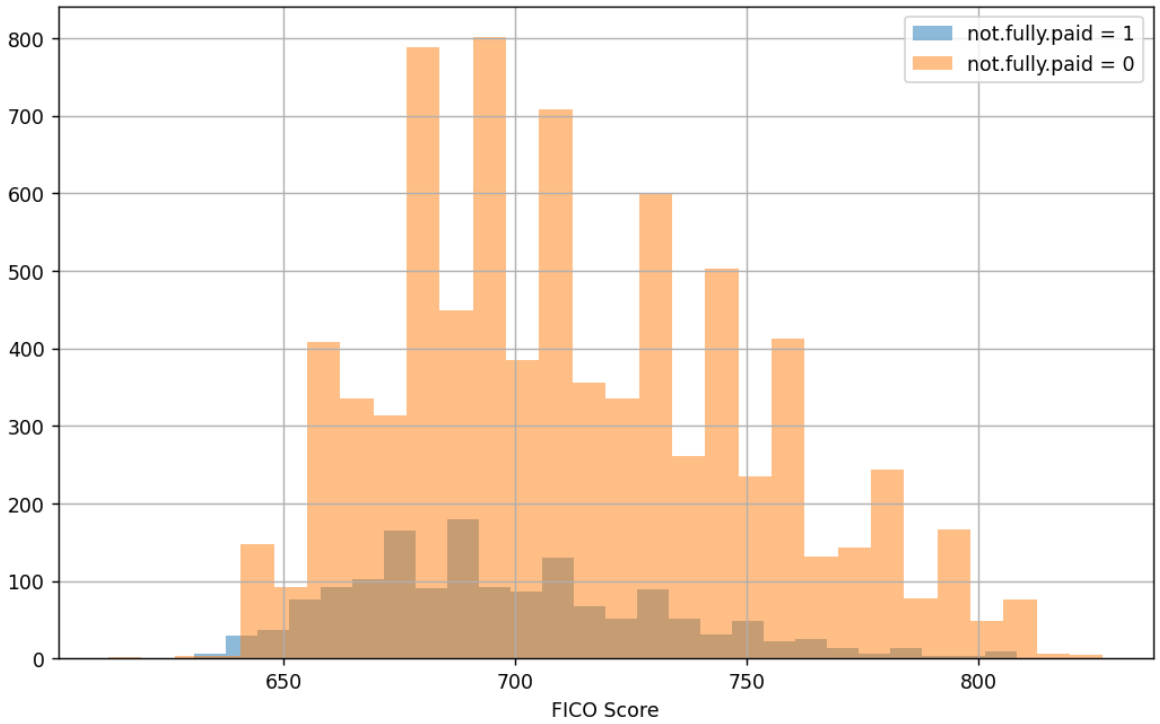
Target value	Meaning	Records
0	Loan fully paid	8,045
1	Loan not fully paid	1,533

2. Exploratory Data Analysis (EDA)

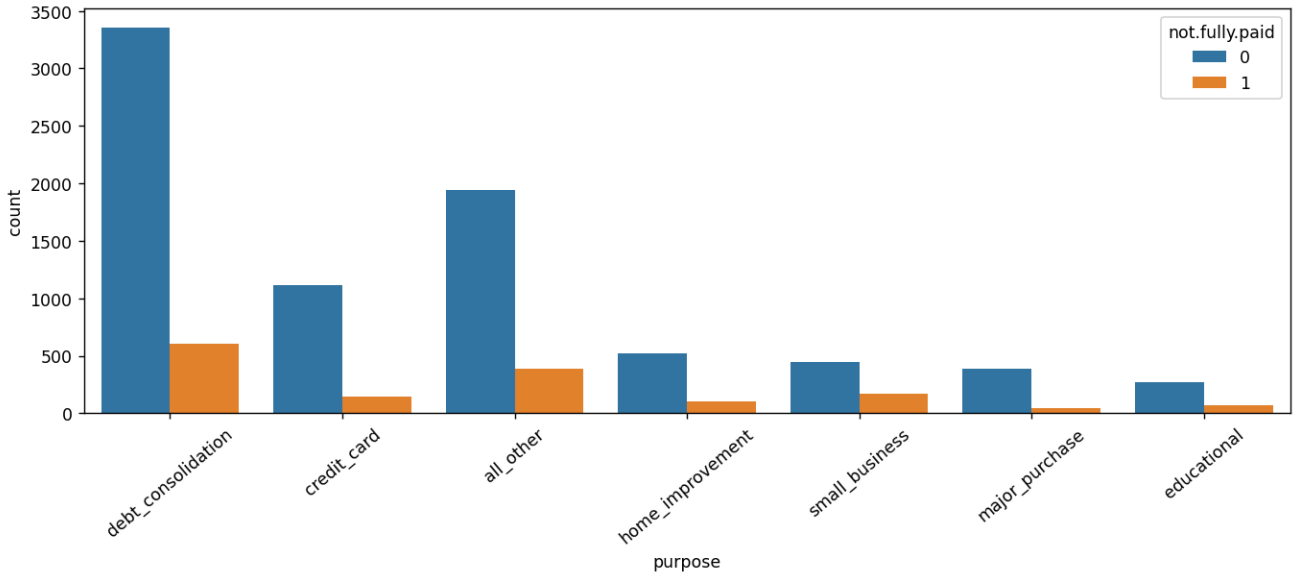
Five plots requested in the project are shown below. They are also saved as outputs within the completed notebook.



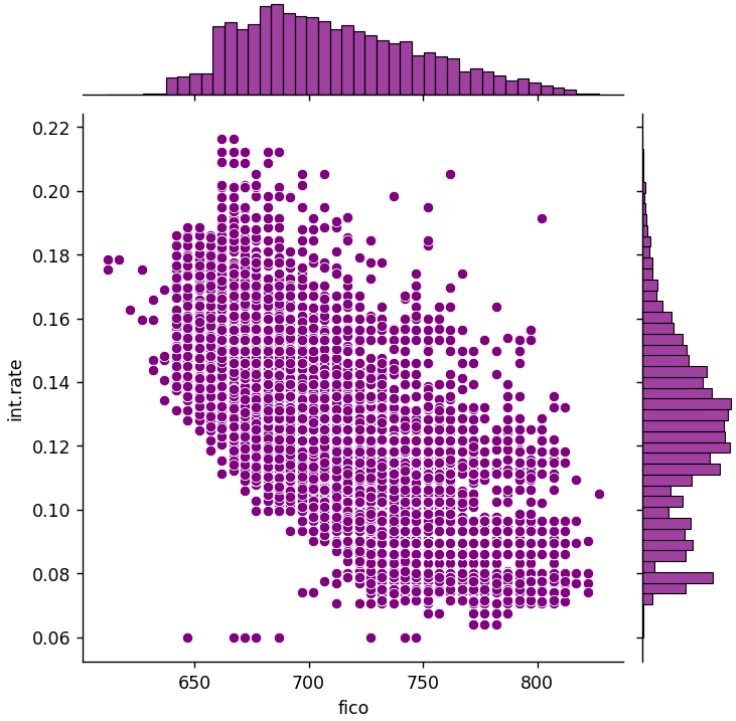
FICO distribution by credit.policy.



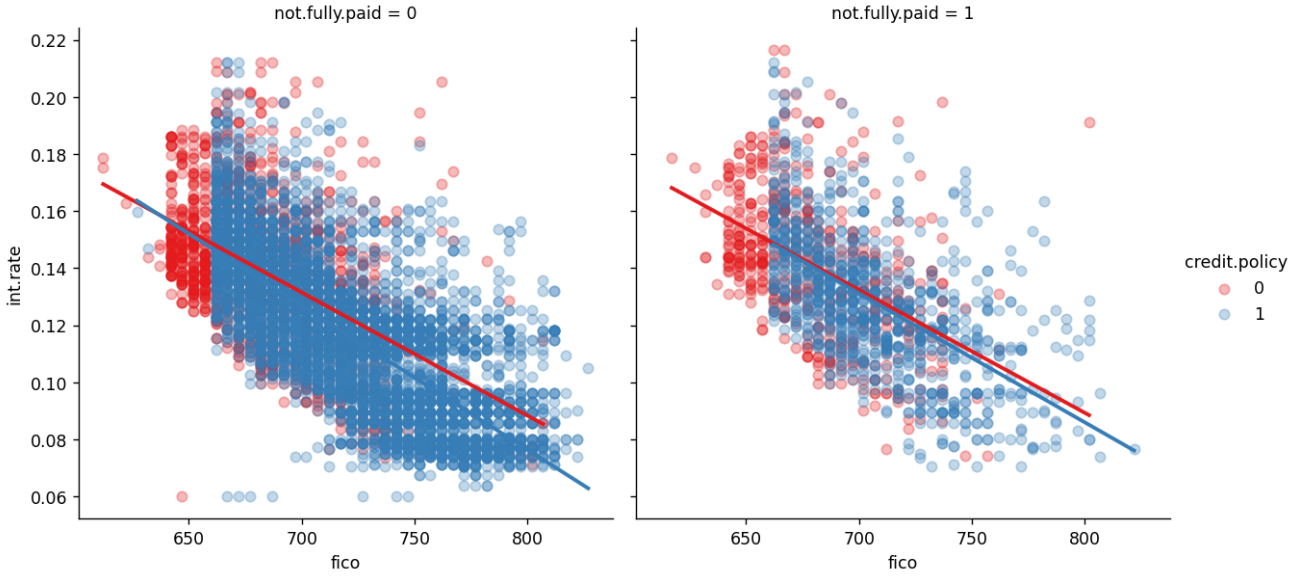
FICO distribution by not.fully.paid.



Loan purpose counts, split by not.fully.paid.



Joint plot comparing FICO scores with interest rates.



FICO vs. interest rate by credit policy and repayment outcome.

3. Prepare categorical features and split the data

```
cat_feats = ['purpose']
final_data = pd.get_dummies(loans, columns=cat_feats, drop_first=True)
X = final_data.drop('not.fully.paid', axis=1)
y = final_data['not.fully.paid']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=101)
```

Training rows: **6,704**. Testing rows: **2,874**. One-hot encoding converts the categorical loan purpose field into numeric columns. The same split is used for both models.

4. Train and evaluate Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
dtree = DecisionTreeClassifier(random_state=101)
```

```
dtree.fit(X_train, y_train)
predictions = dtree.predict(X_test)
print(classification_report(y_test, predictions))
print(confusion_matrix(y_test, predictions))
```

Evaluation	Value
Accuracy	0.7244 (72.44%)
Class 1 recall	0.2415
Confusion matrix	[[1975, 456], [336, 107]]

5. Train and evaluate Random Forest

```
from sklearn.ensemble import RandomForestClassifier
rfc = RandomForestClassifier(n_estimators=100, random_state=101)
rfc.fit(X_train, y_train)
rfc_pred = rfc.predict(X_test)
print(classification_report(y_test, rfc_pred))
print(confusion_matrix(y_test, rfc_pred))
```

Evaluation	Value
Accuracy	0.8462 (84.62%)
Class 1 recall	0.0226
Confusion matrix	[[2422, 9], [433, 10]]

6. Final question: Which model performed better?

The **Random Forest** performed better in **overall accuracy** (84.62%) than the **Decision Tree** (72.44%). However, the Decision Tree achieved better recall for not-fully-paid loans (class 1): 24.15% versus 2.26%. Since the target classes are imbalanced, accuracy alone does not tell the whole story.

The test run produced valid classification reports and confusion matrices. Any variation in results in another run can be avoided by preserving the random_state values shown above.

7. Included submission files

1. Lab9_Decision_Trees_Random_Forest_Solved.ipynb - original project questions filled and executed.
2. Lab9_Decision_Trees_Random_Forest.py - Python version of the solution.
3. Lab9_Solution.pdf - this report.
4. loan_data.csv - original project dataset.

Upload the four files to your GitHub repository and submit the repository link in Blackboard, unless the instructor gives different instructions.